4주차 3일차: 입력 URL과 기본 오류 처리 — 30분

1. 학습 목표 — 2분

- 입력한 URL을 실제 GET 요청에 사용합니다.
- 잘못된 주소, HTTP 실패, JSON 형식 오류, 시간 초과를 상태 문구로 안내합니다.

시작 상태는 day14입니다.

2. 핵심 개념 설명 — 6분

URL을 입력받으면 잘못된 형식이 들어올 수 있습니다. Uri.TryCreate는 문자열이 절대 주소로 해석되는지 확인합니다. 그다음 http 또는 https인지 확인합니다. 이는 주소 형식 확인이며 서버가 존재하거나 응답 형식이 맞는지까지 보장하지 않습니다.

성공하면 생성한 uri를 GetFromJsonAsync에 넘깁니다. 호출을 시작할 때 기존 행을 비워 실패 후에도 이전 결과가 새 결과처럼 보이지 않게 합니다. 0건을 나타내는 빈 배열 []는 정상 결과이고, JSON null이나 객체 하나는 이 프로그램이 기대한 배열이 아닙니다.

상황	처리
"잘못된주소"나 file 주소	요청 전에 상태 안내 후 return
연결 실패 또는 HTTP 실패 상태	HttpRequestException 처리
JSON 문법·형식 불일치	JsonException 처리
15초 제한 초과	OperationCanceledException 처리

AsyncRelayCommand의 기본 설정은 예외를 조용히 삼키지 않습니다. 예상 가능한 요청 오류는 RunAsync 안에서 catch합니다. 여기에는 취소 버튼이 없으므로 OperationCanceledException은 시간 초과 안내로 사용합니다.

WPF 버튼에서 호출된 이 메서드는 기본 await 동작으로 UI 컨텍스트에 이어집니다. 따라서 await 뒤에 Posts를 채우는 데 별도 Task.Run이나 Dispatcher 코드를 넣지 않습니다. UI 스레드와 다른 스레드에서 컬렉션을 수정하는 고급 주제는 제외합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
        if (!Uri.TryCreate(Input.Trim(), UriKind.Absolute, out Uri? uri)
            || (uri.Scheme != Uri.UriSchemeHttp && uri.Scheme != Uri.UriSchemeHttps))
        {
            Status = "http 또는 https로 시작하는 API URL을 입력해 주세요.";
            return;
        }
```

요청 부분은 고정 문자열 대신 uri를 사용합니다.

```
        List<Post>? posts = await client.GetFromJsonAsync<List<Post>>(
            uri);
```

기본 오류 처리 중 HTTP 실패 안내는 다음과 같습니다. 전체 파일에는 JSON 오류와 시간 초과 catch도 이어집니다.

```
    catch (HttpRequestException)
    {
        Status = "요청에 실패했습니다. URL, 인터넷 연결, HTTP 응답 상태를 확인하세요.";
    }
```

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainWindow.xaml	day15의 MainWindow.xaml
MainViewModel.cs	day15의 MainViewModel.cs

4. Visual Studio에서 따라 하는 실습 — 14분

1. **3분:** 실행을 종료하고 MainViewModel.cs와 MainWindow.xaml을 전체 코드로 교체합니다. 입력창 위 안내가 실제 URL 입력으로 바뀌었는지 확인합니다.
2. **4분:** F5로 실행합니다. 기본 주소로 조회한 후 userId=1을 userId=2로 바꾸고 조회합니다. 다른 작성자 데이터가 나타나는지 확인합니다.
3. **3분:** "잘못된주소"를 입력하고 실행합니다. 요청 전에 안내가 표시되고 기존 표는 비워집니다. 빈 입력에서는 버튼이 비활성화되는 것도 확인합니다.
4. **4분:** 주소를 https://jsonplaceholder.typicode.com/posts/1 로 바꾸어 실행합니다. 이는 단일 객체이므로 배열 형식 오류 안내가 나타납니다. 기본 주소로 복구해 다시 성공하는지 확인하고 종료합니다.

시간 초과는 네트워크 환경에 따라 재현 시간이 달라 일부러 만들지 않아도 됩니다. 코드에서 15초 설정과 해당 catch의 위치만 확인합니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 14로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 15로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- 입력 URL의 형식 확인과 실제 요청 성공은 별개의 문제입니다.
- 예상 가능한 오류는 catch하여 상태 문구로 알립니다.
- 실패 후에는 이전 결과를 표시하지 않고, 이후 정상 요청을 다시 실행할 수 있습니다.
- API 응답 모양에 맞는 C# 클래스가 필요하며 임의 JSON을 자동으로 표로 바꾸지는 않습니다.

완료본: day15. 다음: 4주차 4일차.